

TYPES OF DATA FILES

1. CSV, Excel, JSON, XML, TXT

Think of files as **different types of containers** for storing information. They all store data, but each does it in its own way.

File Type	Imagine It As	Best For
CSV	A simple table	Lists of data
Excel (.xlsx)	A smart notebook with tables	Reports, calculations
JSON	A dictionary	Apps and websites
XML	A tagged document	Sharing structured information
TXT	Plain paper	Notes or simple text

1. CSV (Comma-Separated Values)

A CSV file stores data like a table.

Example:

```
Name, Age, City
Alice, 15, Delhi
Bob, 16, Mumbai
Charlie, 14, Surat
```

Think of it like:

Name	Age	City
------	-----	------

Alice	15	Delhi
-------	----	-------

Bob	16	Mumbai
-----	----	--------

Advantages

- Very small file size
- Easy to read
- Works with Excel
- Easy to use in Python

Disadvantages

- No colors
- No formulas
- No images

Use Cases

- Student records
- Employee lists
- Sales data
- Machine learning datasets

2. Excel (.xlsx)

Excel is like a powerful spreadsheet.

Example:

Name	Marks	Total
Alice	80	=SUM(B2)

Unlike CSV, Excel can have:

- Colors
- Charts
- Multiple sheets
- Formulas
- Images

Advantages

- Beautiful formatting
- Automatic calculations
- Charts
- Multiple worksheets

Disadvantages

- Larger file size
- Harder to process than CSV

Use Cases

- Office reports
- School marksheets
- Business accounting
- Budget planning

3. JSON (JavaScript Object Notation)

JSON stores data as **key-value pairs**.

Example:

```
{  
  "name": "Alice",  
  "age": 15,  
  "city": "Delhi"  
}
```

Think of it like:

Name → Alice
Age → 15
City → Delhi

Advantages

- Easy for computers
- Easy for Python
- Used by websites
- Lightweight

Disadvantages

- Not great for viewing as tables
- Can become complex

Use Cases

- APIs
 - Mobile apps
 - Web applications
 - Configuration files
-

4. XML (Extensible Markup Language)

XML uses opening and closing tags.

Example:

```
<student>
  <name>Alice</name>
  <age>15</age>
  <city>Delhi</city>
</student>
```

Think of it as labeled boxes.

Advantages

- Organized
- Good for large systems
- Easy to validate

Disadvantages

- Longer than JSON
- More typing

Use Cases

- Banking
- Government systems
- Medical records
- Data exchange

5. TXT (Text File)

A TXT file is just plain text.

Example:

```
Hello
My name is Alice.
I love Python.
```

Advantages

- Very simple
- Small
- Opens everywhere

Disadvantages

- No tables
- No formatting
- No formulas

Use Cases

- Notes
- Logs
- ReadMe files
- Simple data

Which should you use?

- **CSV:** Large tables and datasets
- **Excel:** Reports and calculations
- **JSON:** Web and app data exchange
- **XML:** Enterprise systems and structured data sharing
- **TXT:** Plain text, notes, and logs

File Handling Basics in Python

File handling means **reading from or writing to files** using Python.

Imagine a file is a notebook.

- Open the notebook
- Read or write
- Close the notebook

Python can do all of this.

Opening a File

```
file = open("data.txt", "r")
```

- "data.txt" → file name
- "r" → read mode

Reading a File

Suppose `data.txt` contains:

```
Hello  
Welcome to Python
```

Code:

```
file = open("data.txt", "r")  
  
content = file.read()  
  
print(content)  
  
file.close()
```

Output:

```
Hello  
Welcome to Python
```

Writing to a File

```
file = open("data.txt", "w")  
  
file.write("Hello World")  
  
file.close()
```

Now the file contains:

```
Hello World
```

Note: "w" (write mode) replaces all existing content in the file.

Appending to a File

Append means **add new content without deleting existing content**.

```
file = open("data.txt", "a")  
  
file.write("\nPython is fun.")  
  
file.close()
```

If the file already had:

```
Hello World
```

It becomes:

```
Hello World  
Python is fun.
```

Using `with` (Recommended)

Python automatically closes the file when using `with`.

```
with open("data.txt", "r") as file:  
    content = file.read()  
  
print(content)
```

This is the preferred way because it safely handles closing the file.

File Modes

Mode	Meaning
"r"	Read an existing file
"w"	Write (overwrite existing content)
"a"	Append (add to the end)
"x"	Create a new file (fails if it already exists)
"rb"	Read a binary file (e.g., images)
"wb"	Write a binary file

Simple Example

Suppose `students.txt` contains:

```
Alice
Bob
Charlie
```

Python code:

```
with open("students.txt", "r") as file:
    print(file.read())
```

Output:

```
Alice
Bob
Charlie
```